

CLOUD SEC & IA

PYTHON TO MACHINE LEARNING II



15

LISTA DE FIGURAS

Figura 1 – Indexação de Lista	8
Figura 2 – Informações estatísticas (2)	10
Figura 3 – Potenciação e função logarítmica	10
Figura 4 – Listas de colunas do DataFrame	11
Figura 5 – Funções estatísticas do DataFrame	12
Figura 6 – Método sort_values	12
Figura 7 – Método sort com duas colunas e ordem decrescente	13
Figura 8 – Usando .loc	13
Figura 9 – Critérios condicionais	14
Figura 10 – Critérios condicionais com and (&)	14
Figura 11 – Fazendo a leitura de um arquivo .csv	15
Figura 12 – Comando para compartilhar seu Google Drive com o Colab	15
Figura 13 – Clicando no link	16
Figura 14 – Tela com o Token	16
Figura 15 – Mounted at	16
Figura 16 – Método head	17
Figura 17 – Método .tail	17
Figura 18 – Método .unique	18
Figura 19 – Método .value_counts	18
Figura 20 – Método .value_counts(normalize=True)	18
Figura 21 – Método .groupby	19
Figura 22 – Método .groupby, mean e sort_values	19
Figura 23 – Método .apply lambda	21
Figura 24 – Método .replace	22
Figura 25 – Método .dropna	22
Figura 26 – Método .fillna	23
Figura 27 – Método .isna	23
Figura 28 – Montando dois DataFrames	25
Figura 29 – Método .merge	25
Figura 30 – Sql	26
Figura 31 – Left_on right_on	26
Figura 32 – Right_index	26
Figura 33 – how='left'	27
Figura 34 – Left Sql equivalente	27
Figura 35 – how='right'	28
Figura 36 – Right Sql equivalente	28
Figura 37 – Full outer join	29
Figura 38 – Full Join Sql	29
Figura 39 – Método .concat	30
Figura 40 – Parâmetro axis	30
Figura 41 – Split-apply-combine	31
Figura 42 – Leitura do arquivo convertendo a string de salário	32
Figura 43 – Groupby e count	33
Figura 44 – Size	33
Figura 45 – Sum, mean e median	34
Figura 46 – Cinco departamentos com mais cargos	35

Figura 47 – SQL funcionários mais bem pagos de cada departamento	35
Figura 48 – Ranking	36

EMAP

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Import.....	8
Código-fonte 2 – Criando um array	8
Código-fonte 3 – Exibindo o array	9
Código-fonte 4 – Apresentando o índice ao array	9
Código-fonte 5 – Atribuindo um índice ao array	9
Código-fonte 6 – Acessar o valor pelo index.....	9
Código-fonte 7 – Informações estatísticas	9
Código-fonte 8 – Potenciação	10
Código-fonte 9 – Função logarítmica	10
Código-fonte 10 – Criando e exibindo um DataFrame	11
Código-fonte 11 – Exibindo os tipos de dados das colunas do DataFrame	11
Código-fonte 12 – Listas de colunas do DataFrame	11
Código-fonte 13 – Notas de seminário do DataFrame	11
Código-fonte 14 – Sort do DataFrame com duas colunas.....	13
Código-fonte 15 – Sort decrescente do DataFrame	13
Código-fonte 16 – Usando .loc	13
Código-fonte 17 – Critérios condicionais com and (&)	14
Código-fonte 18 – Exemplo de .csv em Google Drive.....	15
Código-fonte 19 – Comando para compartilhar seu Google Drive com o Colab	16
Código-fonte 20 – head com o .csv em uma pasta no Google Drive	16
Código-fonte 21 – . head com quantidade de linhas	17
Código-fonte 22 – .head com quantidade de linhas	17
Código-fonte 23 – Método .tail	17
Código-fonte 24 – Método .unique	18
Código-fonte 25 – Método .value_counts.....	18
Código-fonte 26 – Método .value_counts(normalize=True).....	18
Código-fonte 27 – Método .groupby	19
Código-fonte 28 – Método .groupby, mean e sort_values	19
Código-fonte 29 – Dicionário exemplo – lambda.py	20
Código-fonte 30 – usuarios_maior_27.py.....	21
Código-fonte 31 – usuarios_maior_27_lambda.py	21
Código-fonte 32 – Método .apply lambda.....	21
Código-fonte 33 – Método .replace	22
Código-fonte 34 – Método .dropna.....	22
Código-fonte 35 – Método .fillna.....	23
Código-fonte 36 – Método .isna	23
Código-fonte 37 – Novo import.....	24
Código-fonte 38 – Montando dois DataFrames	25
Código-fonte 39 – Método .merge.....	25
Código-fonte 40 – Right_index	26
Código-fonte 41 – how='left'	27
Código-fonte 42 – how='right'	28
Código-fonte 43 – how='outer'	29
Código-fonte 44 – Método .concat	30
Código-fonte 45 – Parâmetro axis.....	30
Código-fonte 46 – Leitura do arquivo convertendo a string de salário	32
Código-fonte 47 – Groupby e count	33

Código-fonte 48 – Size	34
Código-fonte 49 – Sum, mean e median	34
Código-fonte 50 – Cinco departamentos com mais cargos	35
Código-fonte 51 – Classificando os funcionários mais bem pagos	36
Código-fonte 52 – Dez melhores salários por departamento	36
Código-fonte 53 – Cinco melhores salários da UPE	37

EMANIP

SUMÁRIO

INTRODUÇÃO A PYTHON PARA ANÁLISE DE DADOS.....	7
1 INTRODUÇÃO	7
2 SERIES	8
3 DATAFRAME	11
4 LEITURA E MANIPULAÇÃO DE ARQUIVOS	14
4.1 DataFrame Avançado	23
4.2 Joining	24
4.2.1 Inner Join (default)	25
4.2.2 Left Join	27
4.2.3 Right Join	27
4.2.4 Full Outer Join	28
4.3 Combining	29
4.4 Grouping.....	31
4.5 Split-apply-combine	35
CONCLUSÃO.....	37
REFERÊNCIAS.....	38

INTRODUÇÃO A PYTHON PARA ANÁLISE DE DADOS

1 INTRODUÇÃO

Pandas é uma biblioteca de software criada para a linguagem Python voltada à manipulação e análise de dados que oferece estruturas e operações para manipular tabelas numéricas, painéis e séries temporais. É um pacote muito poderoso à disposição do cientista de dados, inclusive para exploração de dados, testes de hipóteses e aprendizagem de máquina.

O Pandas possui dois tipos de estrutura de dados: uma que se assemelha à lista e aos dicionários do Python, com o nome de Pandas DataFrames; e outra que equivale a uma lista do Python, chamada Panda Series ou Pseries, uma estrutura unidimensional que representa cada coluna de um DataFrame.

Este capítulo foi adaptado da documentação oficial do Pandas e tem como objetivo fornecer, de forma enxuta e simplificada, uma apresentação básica dos principais recursos fornecidos pelo Pandas.

Vamos começar pela importação das bibliotecas para o Pandas. Uma delas é o NumPy uma poderosa biblioteca Python usada principalmente para realizar cálculos em Arrays Multidimensionais.

O NumPy fornece um grande conjunto de funções e operações de biblioteca que ajudam os programadores a executar facilmente cálculos numéricos. Esses tipos de cálculos numéricos são amplamente utilizados em tarefas como modelos de *Machine Learning*, processamento de imagem e computação gráfica; e tarefas matemáticas, como integração numérica, diferenciação, interpolação, extrapolação, entre muitas outras.

O Matplotlib é uma biblioteca de software para a criação de gráficos e visualizações de dados em geral feita para a linguagem de programação Python e sua extensão de matemática NumPy. No código fonte “Import”, apresentamos como carregar as bibliotecas no *python*.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline
```

Código-fonte 1 – Import
Fonte: Elaborado pelo autor (2021)

Como apresentado no início deste capítulo, existem dois tipos principais de estruturas de dados no Pandas:

- Series
- DataFrame

2 SERIES

Uma Series é como um array unidimensional, uma lista de valores. Toda Series possui um índice, o index, que dá rótulos a cada elemento da lista.

Na figura “Indexação de lista”, é resumido o processo de criação da Serie e como obter os dados. Nos códigos fonte “Criando um array”, “Exibindo o array” e “Apresentando o índice ao array” podemos detalhar o processo.

```
[ ] notas = pd.Series([2,7,5,10,6])
notas

0      2
1      7
2      5
3     10
4      6
dtype: int64

[ ] notas.values

array([ 2,  7,  5, 10,  6])

[ ] notas.index
#começa no index -> 0 acaba no index -> 5 com incremento de 1

RangeIndex(start=0, stop=5, step=1)
```

Figura 1 – Indexação de Lista
Fonte: Elaborado pelo autor (2021)

```
notas = pd.Series([2,7,5,10,6])
notas
```

Código-fonte 2 – Criando um array

Fonte: Elaborado pelo autor (2021)

```
notas.values
```

Código-fonte 3 – Exibindo o array
Fonte: Elaborado pelo autor (2021)

```
notas.index
```

Código-fonte 4 – Apresentando o índice ao array
Fonte: Elaborado pelo autor (2021)

Como ao criar a Series não demos um índice específico, o Pandas usou inteiros positivos crescentes como padrão. Pode ser conveniente atribuímos um índice diferente do padrão. Supondo que essas sejam notas de uma turma, poderíamos atribuir nomes ao index, conforme podemos observar no código fonte “Atribuindo um índice ao array”:

```
notas = pd.Series([2,7,5,10,6], index=["Wilfred", "Abbie",  
"Harry", "Julia", "Carrie"])  
notas
```

Código-fonte 5 – Atribuindo um índice ao array
Fonte: Elaborado pelo autor (2021)

Agora podemos acessar o valor do dado pelo index, conforme podemos observar no código fonte “Acessar o valor pelo index”.

```
notas["Julia"]
```

Código-fonte 6 – Acessar o valor pelo index
Fonte: Elaborado pelo autor (2021)

Outra facilidade proporcionada pela estrutura são seus métodos que fornecem informações estatísticas sobre os valores, como média (.mean()) e desvio-padrão (.std()), conforme observamos no código fonte “Informações estatísticas”.

```
print("Média:", notas.mean())  
print("Desvio padrão:", notas.std())
```

Código-fonte 7 – Informações estatísticas
Fonte: Elaborado pelo autor (2021)

Ao resumir brevemente as estatísticas dos dados, usa-se o `.describe()`, para exibir média, desvio-padrão, mínimo, máximo, entre outros.

```
[ ] notas.describe()

count      5.000000
mean       6.000000
std        2.915476
min        2.000000
25%        5.000000
50%        6.000000
75%        7.000000
max       10.000000
dtype: float64
```

Figura 2 – Informações estatísticas (2)
Fonte: Elaborado pelo autor (2021)

A estrutura é flexível o suficiente para aplicarmos algumas expressões e funções matemáticas do NumPy, como potenciação e função logarítmica diretamente:

```
[ ] notas**2

Wilfred      4
Abbie       49
Harry       25
Julia       100
Carrie       36
dtype: int64

[ ] np.log(notas)

Wilfred      0.693147
Abbie       1.945910
Harry       1.609438
Julia       2.302585
Carrie       1.791759
dtype: float64
```

Figura 3 – Potenciação e função logarítmica
Fonte: Elaborado pelo autor (2021)

```
notas**2
```

Código-fonte 8 – Potenciação
Fonte: Elaborado pelo autor (2021)

```
np.log(notas)
```

Código-fonte 9 – Função logarítmica
Fonte: Elaborado pelo autor (2021)

3 DATAFRAME

DataFrame é uma estrutura bidimensional de dados, como uma planilha. Os tipos de dados que compõem as colunas podem ser verificados por um método próprio, conforme obtemos no código fonte “DataFrame”.

```
df = pd.DataFrame({'Aluno': ["Wilfred", "Abbie", "Harry",  
"Julia", "Carrie"],  
                  'Faltas': [3,4,2,1,4],  
                  'Prova': [2,7,5,10,6],  
                  'Seminário': [8.5,7.5,9.0,7.5,8.0]})  
  
df
```

Código-fonte 10 – Criando e exibindo um DataFrame
Fonte: Elaborado pelo autor (2021)

```
df.dtypes
```

Código-fonte 11 – Exibindo os tipos de dados das colunas do DataFrame
Fonte: Elaborado pelo autor (2021)

Acessando a lista de colunas e as notas de seminário do DataFrame.

```
[ ] df.columns  
      Index(['Aluno', 'Faltas', 'Prova', 'Seminário'], dtype='object')  
  
[ ] df["Seminário"]  
      0      8.5  
      1      7.5  
      2      9.0  
      3      7.5  
      4      8.0  
      Name: Seminário, dtype: float64
```

Figura 4 – Listas de colunas do DataFrame
Fonte: Elaborado pelo autor (2021)

```
df.columns
```

Código-fonte 12 – Listas de colunas do DataFrame
Fonte: Elaborado pelo autor (2021)

```
df["Seminário"]
```

Código-fonte 13 – Notas de seminário do DataFrame
Fonte: Elaborado pelo autor (2021)

.describe() também é uma boa forma de verificar resumidamente a disposição estatística dos dados numéricos, conforme podemos observar na figura “Funções estatísticas do DataFrame”:

```
[ ] df.describe()
```

	Faltas	Prova	Seminário
count	5.00000	5.000000	5.00000
mean	2.80000	6.000000	8.10000
std	1.30384	2.915476	0.65192
min	1.00000	2.000000	7.50000
25%	2.00000	5.000000	7.50000
50%	3.00000	6.000000	8.00000
75%	4.00000	7.000000	8.50000
max	4.00000	10.000000	9.00000

Figura 5 – Funções estatísticas do DataFrame
Fonte: Elaborado pelo autor (2021)

Podemos ordenar os DataFrames por uma determinada coluna pelo método sort_values. Observe que, ao utilizar o método, o dataframe original não é modificado, apresentado na figura “Método sort_values”.

```
df.sort_values(by=['Seminário'])
```

	Aluno	Faltas	Prova	Seminário
1	Abbie	4	7	7.5
3	Julia	1	10	7.5
4	Carrie	4	6	8.0
0	Wilfred	3	2	8.5
2	Harry	2	5	9.0

```
# a utilização do método sort_values, não modifica o Dataframe original
df
```

	Aluno	Faltas	Prova	Seminário
0	Wilfred	3	2	8.5
1	Abbie	4	7	7.5
2	Harry	2	5	9.0
3	Julia	1	10	7.5
4	Carrie	4	6	8.0

Figura 6 – Método sort_values
Fonte: Elaborado pelo autor (2021)

Em colunas com o mesmo resultado, podemos inserir outras colunas para reordenação. Por padrão, a ordem é crescente. Caso queira a ordem inversa, é necessário colocar o argumento, ascending = false.

```
[ ] df.sort_values(by=['Seminário', 'Faltas'])
```

	Aluno	Faltas	Prova	Seminário
3	Julia	1	10	7.5
1	Abbie	4	7	7.5
4	Carrie	4	6	8.0
0	Wilfred	3	2	8.5
2	Harry	2	5	9.0

```
[ ] df.sort_values(by=['Seminário', 'Faltas'], ascending=False)
```

	Aluno	Faltas	Prova	Seminário
2	Harry	2	5	9.0
0	Wilfred	3	2	8.5
4	Carrie	4	6	8.0
1	Abbie	4	7	7.5
3	Julia	1	10	7.5

Figura 7 – Método sort com duas colunas e ordem decrescente
Fonte: Elaborado pelo autor (2021)

```
df.sort_values(by=['Seminário', 'Faltas'])
```

Código-fonte 14 – Sort do DataFrame com duas colunas
Fonte: Elaborado pelo autor (2021)

```
df.sort_values(by=['Seminário', 'Faltas'], ascending=False)
```

Código-fonte 15 – Sort decrescente do DataFrame
Fonte: Elaborado pelo autor (2021)

Muitas vezes, é necessário selecionarmos valores específicos de um DataFrame, seja uma linha, seja uma célula específica. Isso pode ser feito de diversas formas. Vamos nos concentrar nas formas mais comuns de selecionar dados.

Para selecionar pelo index ou rótulo, usamos o método .loc.

```
[ ] df.loc[3]
```

```
Aluno      Julia
Faltas      1
Prova      10
Seminário   7.5
Name: 3, dtype: object
```

Figura 8 – Usando .loc
Fonte: Elaborado pelo autor (2021)

```
df.loc[3]
```

Código-fonte 16 – Usando .loc
Fonte: Elaborado pelo autor (2021)

Para selecionar de acordo com critérios condicionais, usa-se o que se chama de Boolean Indexing.

Suponha que queiramos selecionar apenas as linhas em que o valor da coluna Seminário seja acima de 8.0. Podemos realizar essa tarefa passando a condição diretamente como índice, conforme figura “Critérios condicionais”:

```
df[df['Seminário'] > 8.0]
```

	Aluno	Faltas	Prova	Seminário
0	Wilfred	3	2	8.5
2	Harry	2	5	9.0

Figura 9 – Critérios condicionais
Fonte: Elaborado pelo autor (2021)

Esse tipo de indexação também possibilita checar condições de múltiplas colunas. Diferentemente do que estamos habituados em Python, aqui se usam operadores bitwise, ou seja, (&, |, ~) em vez de (and, or, not), respectivamente.

Suponha que, além de `df["Seminário"] > 8.0`, queiramos que o valor da coluna Prova não seja menor que 3:

```
df[(df['Seminário'] > 8.0) & (df['Prova'] > 3)]
```

	Aluno	Faltas	Prova	Seminário
2	Harry	2	5	9.0

Figura 10 – Critérios condicionais com and (&)
Fonte: Elaborado pelo autor (2021)

```
df[(df['Seminário'] > 8.0) & (df['Prova'] > 3)]
```

Código-fonte 17 – Critérios condicionais com and (&)
Fonte: Elaborado pelo autor (2021)

4 LEITURA E MANIPULAÇÃO DE ARQUIVOS

O Pandas nos fornece uma série de funcionalidades de leitura para os mais diversos formatos estruturais de dados. Experimente a autocompletar de `pd.read_`, entre eles, estão:

- `pd.read_csv`, para ler arquivos .csv, formato comum de armazenar dados de tabelas.
- `pd.read_excel`, para ler arquivos Excel .xlsx.

- `pd.read_html`, para ler tabelas diretamente de um website.

O arquivo `dados_v3.csv` contém dados fictícios sobre preços de apartamentos em 7 bairros da cidade de São Paulo.

Para a leitura dos dados, você pode passar o caminho da pasta onde está o arquivo ou fazer um upload para o seu Google Drive em uma pasta específica, e de lá trabalhar com o arquivo na nuvem com o Google Colab de qualquer lugar. Os processos são observados na Figura “Fazendo a leitura de um arquivo .csv” e código fonte “Exemplo de .csv em Google Drive”, respectivamente.

```
# em um ambiente local basta colocar o caminho da pasta junto com o nome do arquivo
# df = pd.read_csv("C:/Download/dados_v3.csv")
df = pd.read_csv("dados_v3.csv")
df
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	350	1	0.0	1.0	21	Campo Belo	340000	16190.48
1	800	1	0.0	1.0	64	Campo Belo	770000	12031.25
2	674	1	0.0	1.0	61	Campo Belo	600000	9836.07
3	700	1	1.0	1.0	70	Campo Belo	700000	10000.00
4	440	1	0.0	1.0	44	Campo Belo	515000	11704.55
...	...	...	...	...	...	...	...	...
1992	1080	3	1.0	1.0	80	Alphaville	680000	8500.00
1993	750	3	0.0	1.0	82	Alphaville	650000	7926.83
1994	700	3	1.0	1.0	100	Alphaville	629900	6299.00
1995	1850	3	1.0	2.0	166	Alphaville	1600000	9638.55
1996	800	3	1.0	1.0	107	Alphaville	540000	5046.73

1997 rows x 8 columns

Figura 11 – Fazendo a leitura de um arquivo .csv
Fonte: Elaborado pelo autor (2021)

```
# df = pd.read_csv('C:\download\dados_v3.csv')
df = pd.read_csv('dados_v3.csv')
df
```

Código-fonte 18 – Exemplo de .csv em Google Drive
Fonte: Elaborado pelo autor (2021)

Caso você deseje trabalhar no Colab com um arquivo armazenado em uma pasta do seu Google Drive, digite os comandos para liberar o acesso do Colab ao Google Drive.

```
from google.colab import drive

drive.mount("/content/drive", force_remount=True)
```

Figura 12 – Comando para compartilhar seu Google Drive com o Colab
Fonte: Elaborado pelo autor (2021)

```
from google.colab import drive
```

```
drive.mount("/content/drive", force_remount=True)
```

Código-fonte 19 – Comando para compartilhar seu Google Drive com o Colab
Fonte: Elaborado pelo autor (2021)

Após a execução desse comando, uma janela irá abrir semelhante à Figura – Clicando no link”. Clique no link.

Go to [this URL](https://accounts.google.com/o/oauth2/auth?client_id=[um-valor-bem-longo]) in a browser: [https://accounts.google.com/o/oauth2/auth?client_id=\[um-valor-bem-longo\]](https://accounts.google.com/o/oauth2/auth?client_id=[um-valor-bem-longo])
Enter your authorization code:

Figura 13 – Clicando no link
Fonte: Elaborado pelo autor (2021)

Uma janela com um token como na Figura – Tela com o Token” irá aparecer. Recorte o token e cole no local: “Enter your Authorization code:” dentro do Colab.



Figura 14 – Tela com o Token
Fonte: Elaborado pelo autor (2021)

Pronto, a mensagem “Mounted at /content/drive” como mostra a Figura – Mounted at” irá aparecer.

```
from google.colab import drive  
drive.mount("/content/drive", force_remount=True)  
Mounted at /content/drive
```

Figura 15 – Mounted at
Fonte: Elaborado pelo autor (2021)

No caminho passado como exemplo para o `read_csv`, a palavra `Datasets` é apenas uma pasta dentro do Google Drive com o arquivo `dados_v3.csv`. O método `.head` exibe as primeiras cinco linhas de um `DataFrame`.

```
df = pd.read_csv('/content/drive/My Drive/Datasets/dados_v3.csv')  
df.head()
```

Código-fonte 20 – head com o .csv em uma pasta no Google Drive
Fonte: Elaborado pelo autor (2021)

Caso queira exibir mais de 5 linhas, use um número de linhas como parâmetro do `.head`.


```
df.head(n=10)
```

Código-fonte 21 – .head com quantidade de linhas
Fonte: Elaborado pelo autor (2021)

```
[ ] df.head()
    df.head(n=10)
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	350	1	0.0	1.0	21	Campo Belo	340000	16190.48
1	800	1	0.0	1.0	64	Campo Belo	770000	12031.25
2	674	1	0.0	1.0	61	Campo Belo	600000	9836.07
3	700	1	1.0	1.0	70	Campo Belo	700000	10000.00
4	440	1	0.0	1.0	44	Campo Belo	515000	11704.55
5	917	1	1.0	1.0	60	Campo Belo	630000	10500.00
6	850	1	1.0	1.0	65	Campo Belo	740000	11384.62
7	350	1	1.0	1.0	43	Campo Belo	570000	13255.81
8	440	1	1.0	1.0	26	Campo Belo	430000	16538.46
9	510	1	1.0	1.0	42	Campo Belo	500000	11904.76

Figura 16 – Método head
Fonte: Elaborado pelo autor (2021)

O método .tail exibe as últimas 5 linhas do DataFrame.

```
df.head(n=10)
```

Código-fonte 22 – .head com quantidade de linhas
Fonte: Elaborado pelo autor (2021)

```
[ ] #.tail(), que exibe por padrão as últimas 5 linhas do DataFrame:
    df.tail()
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
1992	1080	3	1.0	1.0	80	Alphavile	680000	8500.00
1993	750	3	0.0	1.0	82	Alphavile	650000	7926.83
1994	700	3	1.0	1.0	100	Alphavile	629900	6299.00
1995	1850	3	1.0	2.0	166	Alphavile	1600000	9638.55
1996	800	3	1.0	1.0	107	Alphavile	540000	5046.73

Figura 17 – Método .tail
Fonte: Elaborado pelo autor (2021)

```
df.tail()
```

Código-fonte 23 – Método .tail
Fonte: Elaborado pelo autor (2021)

Você pode verificar quais são os dados únicos de uma coluna utilizando o método unique.

```
[ ] df["bairro"].unique()

array(['Campo Belo', 'Itaim', 'Moema', 'Tatuapé', 'Jardins',
      'Jardim Europa', 'Alphavile'], dtype=object)
```

Figura 18 – Método .unique
Fonte: Elaborado pelo Autor (2021)

```
df.bairro.unique()
```

Código-fonte 24 – Método .unique
Fonte: Elaborado pelo autor (2021)

É possível verificar a homogeneidade da nossa amostra em relação aos bairros, ou seja, verificar quantos registros da nossa amostra existem por bairro. Para isso, podemos utilizar o método disponível `value_counts()`.

```
df["bairro"].value_counts()

Itaim          346
Alphavile      341
Campo Belo     307
Jardins        281
Jardim Europa  280
Tatuapé        237
Moema          205
Name: bairro, dtype: int64
```

Figura 19 – Método .value_counts
Fonte: Elaborado pelo autor (2021)

```
df['bairro'].value_counts()
```

Código-fonte 25 – Método .value_counts
Fonte: Elaborado pelo autor (2021)

Os valores contados também podem ser normalizados para expressar porcentagens:

```
[ ] df["bairro"].value_counts(normalize=True)

Itaim          0.173260
Alphavile      0.170756
Campo Belo     0.153731
Jardins        0.140711
Jardim Europa  0.140210
Tatuapé        0.118678
Moema          0.102654
Name: bairro, dtype: float64
```

Figura 20 – Método .value_counts(normalize=True)
Fonte: Elaborado pelo autor (2021)

```
df['bairro'].value_counts(normalize=True)
```

Código-fonte 26 – Método .value_counts(normalize=True)

Fonte: Elaborado pelo autor (2021)

Agrupar dados é uma tarefa fundamental em todas as ferramentas/linguagens que lidam com DataFrames. No Pandas, podemos utilizar o método `.groupby()`.

Agrupamos o nosso DataFrame pelos valores da coluna "bairro" e, em seguida, aplicamos o `.mean()` para termos um objeto GroupBy com informação das médias agrupadas pelos valores da coluna bairros.

```
df.groupby("bairro").mean()
```

	condominio	quartos	suites	vagas	area	preco	pm2
bairro							
Alphaville	681.175953	2.131965	0.944282	1.143695	81.457478	5.750780e+05	7149.804985
Campo Belo	914.475570	2.107492	1.048860	1.159609	83.837134	1.010614e+06	12034.486189
Itaim	991.861272	2.101156	1.034682	1.080925	101.855491	1.216344e+06	11965.298699
Jardim Europa	1260.010714	2.207143	1.064286	1.164286	91.832143	1.946193e+06	20761.351036
Jardins	1357.120996	2.181495	1.192171	1.220641	100.615658	2.033096e+06	19738.407794
Moema	985.234146	2.058537	1.029268	1.200000	88.497561	1.454571e+06	16511.582780
Tatuapé	619.940928	2.097046	0.970464	1.130802	79.949367	4.788869e+05	6145.624473

Figura 21 – Método `.groupby`
Fonte: Elaborado pelo autor (2021)

```
df.groupby('bairro').mean()
```

Código-fonte 27 – Método `.groupby`
Fonte: Elaborado pelo autor (2021)

Para extrairmos dados de uma coluna desse objeto, como, por exemplo, para obtermos os valores da média do preço do metro quadrado em ordem crescente:

```
[ ] df.groupby("bairro").mean()["area"].sort_values()

bairro
Tatuapé      79.949367
Alphaville   81.457478
Campo Belo   83.837134
Moema        88.497561
Jardim Europa 91.832143
Jardins      100.615658
Itaim        101.855491
Name: area, dtype: float64
```

Figura 22 – Método `.groupby`, `mean` e `sort_values`
Fonte: Elaborado pelo autor (2021)

```
df.groupby('bairro').mean()["area"].sort_values()
```

Código-fonte 28 – Método `.groupby`, `mean` e `sort_values`
Fonte: Elaborado pelo autor (2021)

É comum queremos aplicar uma função qualquer aos dados, ou à parte deles. Neste caso, o Pandas fornece o método `.apply`. Por exemplo, para deixar os nomes dos bairros com apenas as suas quatro primeiras letras.

Podemos também usar uma função lambda (notação mais comum), veja a Figura – Método `.apply` lambda”. Funções lambda nada mais são do que funções anônimas. A diferença de uma função normal é que ela não possui nome. Então, ela acaba sendo definida diretamente, muitas vezes, em uma única linha e serve para fornecer abstrações para problemas complexos, deixando o código menor e menos complexo.

Veja um exemplo abaixo. Imagine que você tenha um dicionário de dados chamado “users” com a seguinte estrutura:

```
# Dicionário de dados
users = [
    {
        "id": 1,
        "name": "Allan",
        "age": 27,
        "profile_picture": "http://...",
        "city": "São Paulo"
    },
    {
        "id": 2,
        "name": "Julie",
        "age": 29,
        "profile_picture": "http://...",
        "city": "Curitiba"
    },
    {
        "id": 3,
        "name": "Pedro",
        "age": 31,
        "profile_picture": "http://...",
        "city": "Rio de Janeiro"
    }
]
```

Código-fonte 29 – Dicionário exemplo – lambda.py
Fonte: Elaborado pelo autor (2021)

Imagine que você precise de uma listagem com as pessoas que têm mais de 27 anos. No exemplo do código-fonte com uma função convencional utilizando `def`, temos:

```
def users_over_27(user):  
    return user["age"] > 27  
  
# Filtramos os usuários utilizando o método filter  
filtered_users = filter(users_over_27, users)  
  
list(filtered_users)
```

Código-fonte 30 – usuarios_maior_27.py
Fonte: Elaborado pelo autor (2021)

No próximo trecho de código-fonte, temos o código simplificado em uma só linha, utilizando lambda. Perceba que podemos fazer tudo dentro de outra função, sem precisar criar uma função nomeada.

```
filtered_users = filter(lambda user: user["age"] > 27, users)  
  
list(filtered_users)
```

Código-fonte 31 – usuarios_maior_27_lambda.py
Fonte: Elaborado pelo autor (2021)

Bom, agora que você conheceu pelo exemplo como é uma função lambda, vamos continuar nosso código e aplicar o lambda em nosso arquivo .csv para deixar os nomes dos bairros com apenas as suas quatro primeiras letras:

```
[ ] df["bairro"].apply(lambda x: x[:4])  
  
0      Camp  
1      Camp  
2      Camp  
3      Camp  
4      Camp  
...  
1992  Alph  
1993  Alph  
1994  Alph  
1995  Alph  
1996  Alph  
Name: bairro, Length: 1997, dtype: object
```

Figura 23 – Método .apply lambda
Fonte: Elaborado pelo autor (2021)

```
df["bairro"].apply(lambda x: x[:4])
```

Código-fonte 32 – Método .apply lambda
Fonte: Elaborado pelo autor (2021)

Uma das tarefas nas quais o Pandas é reconhecidamente poderoso é a habilidade de tratar dados incompletos. Por muitos motivos, pode haver incompletude

no dataset. O `np.nan` é um valor especial definido no NumPy que significa Not a Number. O Pandas preenche células sem valores em um DataFrame lido com `np.nan`.

Vamos criar um novo DataFrame usando as 5 primeiras linhas do nosso original, com o já visto `.head()`.

Abaixo, é usado o `.replace` para substituir um valor específico por um NaN. Vamos substituir o valor 12031.25 por NaN.

```
df2 = df.head()
df2 = df2.replace({"pm2": {12031.25: np.nan}})
df2
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	350	1	0.0	1.0	21	Campo Belo	340000	16190.48
1	800	1	0.0	1.0	64	Campo Belo	770000	NaN
2	674	1	0.0	1.0	61	Campo Belo	600000	9836.07
3	700	1	1.0	1.0	70	Campo Belo	700000	10000.00
4	440	1	0.0	1.0	44	Campo Belo	515000	11704.55

Figura 24 – Método `.replace`
Fonte: Elaborado pelo autor (2021)

```
df2 = df.head()
df2 = df2.replace({"pm2": {12031.25: np.nan}})
df2
```

Código-fonte 33 – Método `.replace`
Fonte: Elaborado pelo autor (2021)

O Pandas simplifica a remoção de quaisquer linhas ou colunas que possuem um `np.nan`. Por padrão, o `.dropna()` retorna as linhas que não contêm um NaN:

```
[ ] df2.dropna()
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	350	1	0.0	1.0	21	Campo Belo	340000	16190.48
2	674	1	0.0	1.0	61	Campo Belo	600000	9836.07
3	700	1	1.0	1.0	70	Campo Belo	700000	10000.00
4	440	1	0.0	1.0	44	Campo Belo	515000	11704.55

Figura 25 – Método `.dropna`
Fonte: Elaborado pelo Autor (2021)

```
df2.dropna()
```

Código-fonte 34 – Método `.dropna`
Fonte: Elaborado pelo autor (2021)

Preencher todos os valores NaN por um outro específico também é bem simples:

```
[ ] df2.fillna(9500)
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	350	1	0.0	1.0	21	Campo Belo	340000	16190.48
1	800	1	0.0	1.0	64	Campo Belo	770000	9500.00
2	674	1	0.0	1.0	61	Campo Belo	600000	9836.07
3	700	1	1.0	1.0	70	Campo Belo	700000	10000.00
4	440	1	0.0	1.0	44	Campo Belo	515000	11704.55

Figura 26 – Método .fillna
Fonte: Elaborado pelo autor (2021)

```
df2.fillna(9500)
```

Código-fonte 35 – Método .fillna
Fonte: Elaborado pelo autor (2021)

Método que indica quais valores de um DataFrame são NaN e quais não são:

```
[ ] df2.isna()
```

	condominio	quartos	suites	vagas	area	bairro	preco	pm2
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	True
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False

Figura 27 – Método .isna
Fonte: Elaborado pelo autor (2021)

```
df2.isna()
```

Código-fonte 36 – Método .isna
Fonte: Elaborado pelo autor (2021)

4.1 DataFrame Avançado

Agora que conhecemos o DataFrame, vamos apresentar o potencial que o Pandas oferece.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.set_option('max_columns', 50)
```

Código-fonte 37 – Novo import
Fonte: Elaborado pelo autor (2021)

4.2 Joining

Vamos partir para uma análise mais profunda para observarmos de onde vem o poder do Pandas!

Geralmente, em uma análise, precisamos fazer merge/join de dados que são armazenados de forma relacional.

Como a cláusula JOIN no SQL, `pandas.merge` permite que dois DataFrames possam ser agrupados por meio de uma ou mais chaves. A função provê uma série de parâmetros (`on`, `left_on`, `right_on`, `left_index`, `right_index`) para que você possa especificar sobre quais colunas ou índices deseja realizar o join.

Por padrão, `pandas.merge` opera como um inner join, que pode ser alterado usando o parâmetro `how`.

A partir do docstring:

`how : {'left', 'right', 'outer', 'inner'}, default 'inner'`

- **left:** usa apenas chaves do frame da esquerda (SQL: left outer join).
- **right:** usa apenas chaves do frame da direita (SQL: right outer join).
- **outer:** usa a união das chaves de ambos os frames (SQL: full outer join).
- **inner:** usa a interseção das chaves de ambos os frames (SQL: inner join).

Veja alguns exemplos de cada um:


```
[ ] left_frame = pd.DataFrame({'key': range(5),
                              'left_value': ['a', 'b', 'c', 'd', 'e']})
right_frame = pd.DataFrame({'key': range(2, 7),
                             'right_value': ['f', 'g', 'h', 'i', 'j']})

print(left_frame)
print('\n')
print(right_frame)
```

	key	left_value
0	0	a
1	1	b
2	2	c
3	3	d
4	4	e

	key	right_value
0	2	f
1	3	g
2	4	h
3	5	i
4	6	j

Figura 28 – Montando dois DataFrames
Fonte: Elaborado pelo autor (2021)

```
left_frame = pd.DataFrame({'key': range(5), 'left_value':
['a', 'b', 'c', 'd', 'e']})
right_frame = pd.DataFrame({'key': range(2, 7), 'right_value':
['f', 'g', 'h', 'i', 'j']})
print(left_frame)
print('\n')
print(right_frame)
```

Código-fonte 38 – Montando dois DataFrames
Fonte: Elaborado pelo autor (2021)

4.2.1 Inner Join (default)

Na figura “Método .merge”, apresentamos o processo para realizar o Join de dois *dataframes*.

```
[ ] print pd.merge(left_frame, right_frame, on='key', how='inner')
```

	key	left_value	right_value
0	2	c	f
1	3	d	g
2	4	e	h

Figura 29 – Método .merge
Fonte: Elaborado pelo autor (2021)

```
pd.merge(left_frame, right_frame, on='key', how='inner')
```

Código-fonte 39 – Método .merge
Fonte: Elaborado pelo autor (2021)

Perceba que perdemos alguns dados de ambos os frames já que algumas chaves não batem. O SQL equivalente:

```
SELECT left_frame.key, left_frame.left_value, right_frame.right_value
FROM left_frame
INNER JOIN right_frame
ON left_frame.key = right_frame.key;
```

Figura 30 – Sql
Fonte: Elaborado pelo Autor (2021)

Visto que as chaves das colunas não são totalmente iguais, nós podemos usar `left_on` e `right_on` como parâmetros para especificar em que campos queremos fazer o join de cada frame.

```
pd.merge(left_frame, right_frame, left_on='left_key', right_on='right_key')
```

Figura 31 – Left_on right_on
Fonte: Elaborado pelo Autor (2021)

Uma alternativa, se nossas chaves fossem índices, seria usar `left_index` ou `right_index`, que aceitam valores True/False. Você pode fazer um mix deles e casar as colunas e índices, como a seguir:

`pd.merge(left_frame, right_frame, left_on='key', right_index=True)`

	key	key_x	left_value	key_y	right_value
0	0	0	a	2	f
1	1	1	b	3	g
2	2	2	c	4	h
3	3	3	d	5	i
4	4	4	e	6	j

Figura 32 – Right_index
Fonte: Elaborado pelo autor (2021)

```
pd.merge(left_frame, right_frame, left_on='key', right_index=True)
```

Código-fonte 40 – Right_index
Fonte: Elaborado pelo autor (2021)

4.2.2 Left Join

Na figura “how = ‘left’”, apresentamos o processo para realizar o Left Join de dois dataframes.

```
[ ] pd.merge(left_frame, right_frame, on='key', how='left')
```

	key	left_value	right_value
0	0	a	NaN
1	1	b	NaN
2	2	c	f
3	3	d	g
4	4	e	h

Figura 33 – how='left'
Fonte: Elaborado pelo autor (2021)

```
pd.merge(left_frame, right_frame, on='key', how='left')
```

Código-fonte 41 – how='left'
Fonte: Elaborado pelo autor (2021)

Nós permanecemos tudo que vem do frame da esquerda, extraindo os valores do frame da direita cujas chaves casam. O valor para right_value é NaN para todas as chaves que não casam. Observe o SQL:

```
SELECT left_frame.key, left_frame.left_value, right_frame.right_value
FROM left_frame
LEFT JOIN right_frame
ON left_frame.key = right_frame.key;
```

Figura 34 – Left Sql equivalente
Fonte: Elaborado pelo autor (2021)

4.2.3 Right Join

Na figura “how='right’”, apresentamos o processo para realizar o Right Join de dois dataframes.

```
[ ] pd.merge(left_frame, right_frame, on='key', how='right')
```

	key	left_value	right_value
0	2	c	f
1	3	d	g
2	4	e	h
3	5	NaN	i
4	6	NaN	j

Figura 35 – how='right'
Fonte: Elaborado pelo autor (2021)

```
pd.merge(left_frame, right_frame, on='key', how='right')
```

Código-fonte 42 – how='right'
Fonte: Elaborado pelo autor (2021)

Veja que, desta vez, nós mantivemos todos os valores do frame da direita com os valores para left_value igual a NaN onde as chaves do frame da direita não se casam com as da esquerda.

Equivalente SQL:

```
SELECT right_frame.key, left_frame.left_value, right_frame.right_value
FROM left_frame
RIGHT JOIN right_frame
ON left_frame.key = right_frame.key;
```

Figura 36 – Right Sql equivalente
Fonte: Elaborado pelo autor (2021)

4.2.4 Full Outer Join

Na figura “Full outer join”, apresentamos o processo para realizar o Full Outer Join de dois dataframes.

```
[ ] pd.merge(left_frame, right_frame, on='key', how='outer')
```

	key	left_value	right_value
0	0	a	NaN
1	1	b	NaN
2	2	c	f
3	3	d	g
4	4	e	h
5	5	NaN	i
6	6	NaN	j

Figura 37 – Full outer join
Fonte: Elaborado pelo autor (2021)

```
pd.merge(left_frame, right_frame, on='key', how='outer')
```

Código-fonte 43 – how='outer'
Fonte: Elaborado pelo autor (2021)

Nós mantivemos tudo o que está em ambos os frames, independentemente de eles estarem ou não isolados. Onde não houve um casamento, os valores correspondentes para aquela chave são NaN.

Equivalente SQL (Embora alguns bancos de dados não permitam FULL JOIN's como MySQL):

```
SELECT IFNULL(left_frame.key, right_frame.key) key
, left_frame.left_value, right_frame.right_value
FROM left_frame
FULL OUTER JOIN right_frame
ON left_frame.key = right_frame.key;
```

Figura 38 – Full Join Sql
Fonte: Elaborado pelo autor (2021)

4.3 Combining

Pandas também provê mecanismos de combinar DataFrames a partir do eixo - pandas.concat. Essa função é equivalente ao SQL UNION, mas, logicamente, há muito mais que se pode fazer com ela.

`Pandas.concat` pega uma lista de series ou DataFrames e retorna uma Series ou DataFrame de objetos concatenados. Perceba que, por causa dessa função, você pode combinar vários objetos em um.

```
[ ] pd.concat([left_frame, right_frame])
```

	key	left_value	right_value
0	0	a	NaN
1	1	b	NaN
2	2	c	NaN
3	3	d	NaN
4	4	e	NaN
0	2	NaN	f
1	3	NaN	g
2	4	NaN	h
3	5	NaN	i
4	6	NaN	j

Figura 39 – Método `.concat`
Fonte: Elaborado pelo autor (2021)

```
pd.concat([left_frame, right_frame])
```

Código-fonte 44 – Método `.concat`
Fonte: Elaborado pelo autor (2021)

Por padrão, a função irá fazer um append dos objetos de forma vertical, combinando as colunas com o mesmo valor. Podemos perceber acima que os valores que não casam irão retornar NaN.

Podemos também concatenar só frames de forma lado a lado com o parâmetro `axis`.

```
[ ] pd.concat([left_frame, right_frame], axis=1)
```

	key	left_value	key	right_value
0	0	a	2	f
1	1	b	3	g
2	2	c	4	h
3	3	d	5	i
4	4	e	6	j

Figura 40 – Parâmetro `axis`
Fonte: Elaborado pelo autor (2021)

```
pd.concat([left_frame, right_frame], axis=1)
```

Código-fonte 45 – Parâmetro `axis`

Fonte: Elaborado pelo autor (2021)

Pandas.concat pode ser usado de diversas formas. Todavia, geralmente, é usado para combinar Series/DataFrames em um único objeto.

4.4 Grouping

Agrupamento em Pandas leva um pouco mais de tempo para entender.

O método groupby do Pandas parte da estratégia básica de separar-aplicar-combinar (split-apply-combine) em análise de dados. Ele é uma boa representação de como você pensa sobre um problema de dados, o que é uma habilidade crítica para qualquer analista/cientista de dados.

Quando atacamos um problema de análise de dados, normalmente, o quebramos em pedaços gerenciáveis, realizamos várias operações sobre cada um desses pedaços e depois reagrupamos todos novamente (isso é o essencial da estratégia split-apply-combine). O groupby é muito bom para esses problemas, como podemos ver na Figura – Split-apply-combine”.

Se você já usou o GROUP BY do SQL ou Pivot Table do Excel, deve estar familiarizado com essa estratégia sem saber.

Assumindo que temos um DataFrame e queremos calcular a média de cada grupo visualmente, a técnica split-apply-combine parece assim:

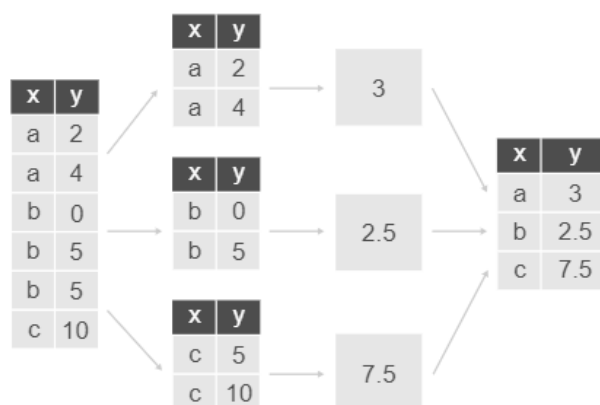


Figura 41 – Split-apply-combine
Fonte: Elaborado pelo autor (2021)

Para explorar as possibilidades dessa metodologia, vamos utilizar uma fonte do portal da transparência. O Recife publica, por meio de Lei de Acesso à Informação,

todos os salários de funcionários públicos da cidade em seu portal de dados. Faça o download do arquivo “salarios_recife_outubro_2013” no repositório da Fase 3. Vamos ilustrar exemplos de groupby com esses dados.

Como os dados nesse arquivo possuem cifrão para cada salário, Python vai tratar cada campo como uma série de strings. Nós podemos usar o parâmetro converter para modificar isso quando ler um arquivo.

Caso tenha optado por colocar seu arquivo em uma pasta do Google Drive, passe o caminho de sua pasta para fazer a leitura do .csv.

```
recife = pd.read_csv('/content/drive/My Drive/Datasets/salarios_recife_outubro_2013.csv',
                    converters={'R$SALARIO': lambda x: float(x.replace('R$', ''))})
recife
```

	ORGAO	NOME	CATEGORIA	CARGO	FUNCAO	R\$SALARIO
0	SECRETARIA DE EDUCACAO	9/16/13	ESTATUTARIO	TECNICO EDUCACIONAL	NaN	1902.24
1	SECRETARIA DE EDUCACAO	GOV. PERNAMBUCO **** CONSISTHR **...	ESTATUTARIO	ASSIST ADMINIST EDUCACIONAL	NaN	2667.97
2	SECRETARIA DE EDUCACAO	ARNEY SPINELI GUIMARAES	ESTATUTARIO	TECNICO EDUCACIONAL	NaN	2302.24
3	SERES - SECRETARIA EXECUTIVA DE RESSOCIALIZACAO	ARNEY SPINELI GUIMARAES	CTD - CONTRATO TEMPO DETERMINADO	PSICOLOGO - CTD	NaN	1770.00
4	SECRETARIA DE EDUCACAO	ITA DE CASSIA FISCHER DEL PINO	ESTATUTARIO	PROFESSOR	NaN	2325.26
...	...	...	...	...	...	...
133907	ITERPE - INSTITUTO DE TERRAS E REFORMA AGRARIA...	SIMONE VALERIA DE GUSMAO SANTOS	CTD - CONTRATO TEMPO DETERMINADO	ASSISTENTE ADMINISTRATIVO CTD	NaN	1100.00
133908	ITERPE - INSTITUTO DE TERRAS E REFORMA AGRARIA...	THAIS SOARES FONSECA	COMISSIONADO	ASSESS SUP AO SEC-ASS NAT TEC	ASSESS SUP AO SEC-ASS NAT TEC	4429.61
133909	ITERPE - INSTITUTO DE TERRAS E REFORMA AGRARIA...	VIVIANE DA SILVA LUCENA PATRIOTA	CTD - CONTRATO TEMPO DETERMINADO	TECNICO NA AREA SOCIAL - CTD	NaN	2200.00
133910	ITERPE - INSTITUTO DE TERRAS E REFORMA AGRARIA...	WAGNER LEITE DE OLIVEIRA	CTD - CONTRATO TEMPO DETERMINADO	TECNICO AGRICOLA - CTD	NaN	1635.00
133911	ITERPE - INSTITUTO DE TERRAS E REFORMA AGRARIA...	WANDERSON SIQUEIRA FREIRE LINS	CTD - CONTRATO TEMPO DETERMINADO	ENGENHEIRO AGRONOMO - CTD	NaN	4590.00

133912 rows x 6 columns

Figura 42 – Leitura do arquivo convertendo a string de salário

Fonte: Elaborado pelo autor (2021)

```
recife = pd.read_csv('/content/drive/My
Drive/Datasets/salarios_recife_outubro_2013.csv',
                    converters={'R$SALARIO': lambda x:
float(x.replace('R$', ''))})
recife
```

Código-fonte 46 – Leitura do arquivo convertendo a string de salário

Fonte: Elaborado pelo autor (2021)

O método groupby do Pandas retorna um objeto DataFrameGroupBy que tem uma variedade de métodos similares às funções-padrão de agregação do SQL.


```
[ ] orgao = recife.groupby('ORGÃO')
    orgao.count()
```

	NOME	CATEGORIA	CARGO	FUNCAO	R\$SALARIO
ORGÃO					
SECRETARIA DE TRABALHO, QUALIFICAÇÃO E EMPREENDEDORISMO	280	280	280	103	280
AD DIPER - AGENCIA DE DESENVOLVIMENTO ECONOMICO DE PERNAMBUCO	143	143	143	104	143
ADMINISTRACAO GERAL DE FERNANDO DE NORONHA	467	467	467	58	467
APAC - AGENCIA PERNAMBUCANA DE AGUAS E CLIMA	97	97	97	92	97
ARPE - AGENCIA ESTADUAL DE REGULACAO DE PERNAMBUCO	84	84	84	77	84
...	...	...	...	...	...
SECRETARIA DO GOVERNO	53	53	53	52	53
SECRETARIA DOS ESPORTES	41	41	41	40	41
SECRETARIA EXTRAORDINARIA DA COPA DE 2014	41	41	41	38	41
SERES - SECRETARIA EXECUTIVA DE RESSOCIALIZACAO	2049	2049	2049	583	2049
UPE - UNIVERSIDADE DE PERNAMBUCO	5115	5115	5115	3312	5115

61 rows × 5 columns

Figura 43 – Groupby e count
Fonte: Elaborado pelo autor (2021)

```
orgao = recife.groupby('ORGÃO')
orgao.count()
```

Código-fonte 47 – Groupby e count
Fonte: Elaborado pelo autor (2021)

Chamando count, ele retorna o número de valores NOT NULL para cada coluna. Se nós estivéssemos interessados no número de registros em cada grupo, poderíamos usar size.

```
[ ] print(orgao.count().head())
    print('\n')
    print(orgao.size().tail())
```

	NOME	CATEGORIA	CARGO	\
ORGÃO				
SECRETARIA DE TRABALHO, QUALIFICAÇÃO E EMPREEND...	280	280	280	
AD DIPER - AGENCIA DE DESENVOLVIMENTO ECONOMICO ...	143	143	143	
ADMINISTRACAO GERAL DE FERNANDO DE NORONHA	467	467	467	
APAC - AGENCIA PERNAMBUCANA DE AGUAS E CLIMA	97	97	97	
ARPE - AGENCIA ESTADUAL DE REGULACAO DE PERNAMBUCO	84	84	84	
FUNCAO R\$SALARIO				
SECRETARIA DE TRABALHO, QUALIFICAÇÃO E EMPREEND...	103	280		
AD DIPER - AGENCIA DE DESENVOLVIMENTO ECONOMICO ...	104	143		
ADMINISTRACAO GERAL DE FERNANDO DE NORONHA	58	467		
APAC - AGENCIA PERNAMBUCANA DE AGUAS E CLIMA	92	97		
ARPE - AGENCIA ESTADUAL DE REGULACAO DE PERNAMBUCO	77	84		
ORGÃO				
SECRETARIA DO GOVERNO	53			
SECRETARIA DOS ESPORTES	41			
SECRETARIA EXTRAORDINARIA DA COPA DE 2014	41			
SERES - SECRETARIA EXECUTIVA DE RESSOCIALIZACAO	2049			
UPE - UNIVERSIDADE DE PERNAMBUCO	5115			

dtype: int64

Figura 44 – Size
Fonte: Elaborado pelo autor (2021)

```
print(orgao.count().head())
print('\n')
print(orgao.size().tail())
```

Código-fonte 48 – Size
Fonte: Elaborado pelo autor (2021)

O somatório pode ser realizado por meio do operador sum, média por mean, etc. (Se existir uma função SQL, há chances de existir em Pandas). Sim e também mediana, que não é encontrada em vários bancos.

```
[ ] print(orgao.sum()[5:10])
    print('\n')
    print(orgao.mean()[5:10])
    print('\n')
    print(orgao.median()[5:10])
```

ORGÃO	R\$SALARIO
ATI - AGENCIA ESTADUAL DE TECNOLOGIA DA INFORMACAO	2931535.84
CEHAB - COMPANHIA ESTADUAL DE HABITACAO E OBRAS	530249.65
CEPE - COMPANHIA EDITORA DA PERNAMBUCO	413150.11
CONDEPE / FIDEM - AGENCIA ESTADUAL DE PLANEJAME...	256889.50
CONSERVATORIO PERNAMBUCANO DE MUSICA	232381.19

ORGÃO	R\$SALARIO
ATI - AGENCIA ESTADUAL DE TECNOLOGIA DA INFORMACAO	5994.960818
CEHAB - COMPANHIA ESTADUAL DE HABITACAO E OBRAS	2995.760734
CEPE - COMPANHIA EDITORA DA PERNAMBUCO	2930.142624
CONDEPE / FIDEM - AGENCIA ESTADUAL DE PLANEJAME...	1902.885185
CONSERVATORIO PERNAMBUCANO DE MUSICA	2799.773373

ORGÃO	R\$SALARIO
ATI - AGENCIA ESTADUAL DE TECNOLOGIA DA INFORMACAO	5821.32
CEHAB - COMPANHIA ESTADUAL DE HABITACAO E OBRAS	2657.77
CEPE - COMPANHIA EDITORA DA PERNAMBUCO	2226.51
CONDEPE / FIDEM - AGENCIA ESTADUAL DE PLANEJAME...	1200.69
CONSERVATORIO PERNAMBUCANO DE MUSICA	2650.71

Figura 45 – Sum, mean e median
Fonte: Elaborado pelo autor (2021)

```
print(orgao.sum() [5:10])
print('\n')
print(orgao.mean() [5:10])
print('\n')
print(orgao.median() [5:10])
```

Código-fonte 49 – Sum, mean e median
Fonte: Elaborado pelo autor (2021)

Operações podem ser feitas também sobre uma Series individual com um objeto grouped. Estamos curiosos para saber os 5 departamentos com mais cargos distintos. O que seria em SQL:

```
[ ] orgao['CARGO'].apply(lambda x: len(x.unique())).sort_values(ascending=False)[:5]
```

ORGAO	
ADMINISTRACAO GERAL DE FERNANDO DE NORONHA	108
SECRETARIA DE SAUDE	65
SECRETARIA DE EDUCACAO	64
HEMOPE - FUNDACAO DE HEMATOLOGIA E HEMOTERAPIA DE PERNAMBUCO	62
UPE - UNIVERSIDADE DE PERNAMBUCO	49
Name: CARGO, dtype: int64	

Figura 46 – Cinco departamentos com mais cargos
Fonte: Elaborado pelo autor (2021)

```
orgao['CARGO'].apply(lambda x:
len(x.unique())).sort_values(ascending=False) [:5]
```

Código-fonte 50 – Cinco departamentos com mais cargos
Fonte: Elaborado pelo autor (2021)

4.5 Split-apply-combine

Vamos conhecer o verdadeiro poder do split-apply-combine com o groupby do Pandas.

Imagine que você deseja saber quem é o funcionário mais bem pago de cada departamento. Olhando nossa base de dados, nós poderíamos fazer algo assim com SQL:

```
SELECT *
FROM recife c
INNER JOIN (
    SELECT ORGAO, max(R$SALARIO) max_salary
    FROM recife
    GROUP BY orgao
) m
ON c.orgao = m.orgao
AND c.R$SALARIO = m.max_salary;
```

Figura 47 – SQL funcionários mais bem pagos de cada departamento
Fonte: Elaborado pelo autor (2021)

Isso nos daria o funcionário mais bem pago de cada órgão, mas também retornaria múltiplo se tivéssemos vários funcionários com o mesmo maior salário em um órgão.

Poderíamos alterar a tabela, adicionar uma coluna e depois atualizar o comando para atualizar aquela coluna, mas sabemos que nem sempre é uma opção.

Usando `groupby`, nós podemos definir uma função que chamaremos de `ranking`. Ela irá rotular cada registro de 1 a N, onde N é o número de empregados em cada órgão. Nós podemos também chamar para aplicar a função para cada grupo, neste caso, cada órgão.

```
[ ] def ranker(df):
    """Atribui uma classificação a cada funcionário com base no salário, sendo 1 o mais bem pago.
    Assume que os dados estão classificados com DESC."""
    df['orgao_rank'] = np.arange(len(df)) + 1
    return df

[ ] recife.sort_values('R$SALARIO', ascending=False, inplace=True)
recife = recife.groupby('ORGAO').apply(ranker)
recife[recife.orgao_rank == 1].head(10)
```

	ORGAO	NOME	CATEGORIA	CARGO	FUNCAO	R\$SALARIO	orgao_rank
61183	SECRETARIA DA FAZENDA	MARIA FIGUEIRA PONTUAL	ESTATUTARIO	AUD FISCAL DO TESOIRO ESTADUAL	NaN	41065.84	1
40997	PROCURADORIA GERAL DO ESTADO	JOSE GAUDENCIO FILHO	ESTATUTARIO	PROCURADOR PE IV	NaN	39775.44	1
115281	IPA - INSTITUTO AGRONOMICO DE PERNAMBUCO	ELIANE DE CARVALHO NOYA	CELETISTA	PESQUISADOR	NaN	32887.15	1
29462	SECRETARIA DE DEFESA SOCIAL	GENIVALDO NASCIMENTO DE MELO	ESTATUTARIO	DELEGADO DE POLICIA	NaN	31967.23	1
131418	CORPO DE BOMBEIROS MILITAR DE PERNAMBUCO	MAWIAEL REIMINE DA SILVA	MILITAR	CORONEL	NaN	30796.57	1
91318	ATI - AGENCIA ESTADUAL DE TECNOLOGIA DA INFORM...	ADEMILTON CIPRIANO DA SILVA	CELETISTA	AM EM GEST TEC INF E COM-SUP	NaN	28953.55	1
95228	POLICIA MILITAR DE PERNAMBUCO	CARLOS ALBERTO NASCIMENTO FEITOSA	MILITAR	CORONEL	NaN	27155.08	1
68923	SECRETARIA DE SAUDE	MICHELLINI JANUNCIO DE ALMEIDA	ESTATUTARIO	MEDICO MEDICO PED I	NaN	25640.42	1
126288	FUNASE - FUNDACAO DE ATENDIMENTO SOCIOEDUCATIVO	JAIME CESAR DE ALBUQUERQUE	CELETISTA	ANAL EM GESTAO AUT/FUND ANGAF	NaN	19644.96	1
133410	PERPART - PERNAMBUCO PARTICIPACOES E INVESTIME...	URIAS DE MENEZES FEITOSA	CELETISTA	ANALISTA DE DESENVOLVIMENTO	NaN	19196.61	1

```
[ ] recife[recife.ORGAO == "UPE - UNIVERSIDADE DE PERNAMBUCO"][:5]
```

	ORGAO	NOME	CATEGORIA	CARGO	FUNCAO	R\$SALARIO	orgao_rank
123292	UPE - UNIVERSIDADE DE PERNAMBUCO	PEDRO DE ALCANTARA NETO	ESTATUTARIO	PROFESSOR ADJUNTO	DIRETOR DE UNIDADE	17996.75	1
119966	UPE - UNIVERSIDADE DE PERNAMBUCO	CARLOS FERNANDO DE ARAUJO CALADO	ESTATUTARIO	PROFESSOR ADJUNTO	REITOR	17645.02	2
120435	UPE - UNIVERSIDADE DE PERNAMBUCO	EDLA CRISTINA ARAUJO MONTEIRO DA CRUZ	ESTATUTARIO	MEDICO	SUPERVISOR III	17045.32	3
122416	UPE - UNIVERSIDADE DE PERNAMBUCO	MARIA DA SALETE FONSECA DOS SANTOS LUNDGREN	ESTATUTARIO	MEDICO	SUPERVISOR III	16821.23	4
119833	UPE - UNIVERSIDADE DE PERNAMBUCO	ARONITA ROSENBLATT	ESTATUTARIO	PROFESSOR TITULAR	NaN	16711.02	5

Figura 48 – Ranking
Fonte: Elaborado pelo autor (2021)

```
def ranker(df):
    """Atribui uma classificação a cada funcionário com base no salário,
    sendo 1 o mais bem pago.
    Assume que os dados estão classificados com DESC."""
    df['orgao_rank'] = np.arange(len(df)) + 1
    return df
```

Código-fonte 51 – Classificando os funcionários mais bem pagos
Fonte: Elaborado pelo autor (2021)

```
recife.sort_values('R$SALARIO', ascending=False, inplace=True)
recife = recife.groupby('ORGAO').apply(ranker)
recife[recife.orgao_rank == 1].head(10)
```

Código-fonte 52 – Dez melhores salários por departamento
Fonte: Elaborado pelo autor (2021)

Agora vimos o ranking de empregados em cada órgão baseado no salário. Podemos também ranquear os funcionários de apenas um órgão, informando o nome do órgão e definindo a quantidade de itens que irá aparecer, como por exemplo: top cinco salários da Universidade de Pernambuco.

```
recife[recife.ORGAO == "UPE - UNIVERSIDADE DE PERNAMBUCO"] [:5]
```

Código-fonte 53 – Cinco melhores salários da UPE
Fonte: Elaborado pelo autor (2021)

Gostou do Pandas? Imagine, ao ficar expert nesse tipo de conteúdo, quantas informações importantes consegue extrair utilizando o Pandas e a linguagem Python.

Revise e brinque com os arquivos csv's. Além de procurar tantos outros que estão disponíveis pela Internet, seja um investigador e busque, com esses dados brutos, levantar informações relevantes. Esse tipo de treinamento irá fazer com que no futuro você seja um cientista de dados muito competente. Até a próxima fase!

CONCLUSÃO

Com este capítulo, conseguimos mostrar o poder do Pandas juntamente ao Python para a análise de dados. Faça outros testes e busque mais informações dentro dos arquivos .csv que foram disponibilizados. Lembre-se de que o sucesso depende de sua prática nos conteúdos. Portanto, não deixe de explorar as bases e buscar outras saídas para os dados.

REFERÊNCIAS

BRAGA, Otávio. Entendendo as funções Lambda no Python. **Medium**, 5 jun. 2018. Disponível em: <<https://medium.com/@otaviobn/entendendo-as-funcoes-lambda-no-python-cbe3c5abb179>>. Acesso em: 23 mar. 2021.

DOWNEY, Allen. **Pense em Python**: pense como um cientista da computação. São Paulo: Novatec, 2016.

GRUS, Joel. **Data Science do zero**: primeiras regras com o Python. Rio de Janeiro: Alta Books, 2019.

MCKINNEY, Wes. **Python para análise de dados**: tratamento de dados com Pandas, NumPy e IPython. São Paulo: Novatec, 2019.

MENEZES, Nilo Ney Coutinho. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. São Paulo: Novatec, 2010.

RAMALHO, Luciano. **Python fluente**: programação clara, concisa e eficaz. Novatec, 2015.

SANTIAGO JR., Luiz. Entendendo a biblioteca NumPy. **Medium, Ensina.AI**, 30 sep. 2018. Disponível em: <<https://medium.com/ensina-ai/entendendo-a-biblioteca-numpy-4858fde63355>>. Acesso em: 23 mar. 2021.